

VALOMETRICS

— INFORME DEL PROYECTO

ARQUITECTURA DE SOFTWARE
FastAPI + React + PostgreSQL + Docker



Autor: David Andrés Díaz García

Profesor: Aldemír Vargas Eudor

MAYO 2026

1. Introducción

1.1. Descripción del problema

Los jugadores competitivos de Valorant (Riot Games) no cuentan con una herramienta personalizada, gratuita y de código abierto que les permita centralizar sus estadísticas de rendimiento, visualizar su progreso a lo largo del tiempo y detectar patrones de mejora. Si bien existen plataformas como Tracker.gg, Blitz.gg o Mobalytics, estas son aplicaciones genéricas que no siempre se adaptan a las necesidades específicas de cada jugador ni permiten la personalización ni el análisis offline de los datos.

1.2. Justificación de la solución

ValoMetrics nace como una plataforma web de análisis de estadísticas diseñada específicamente para jugadores de Valorant. La solución permite:

- Centralizar las estadísticas de juego en un solo lugar.
- Visualizar el rendimiento mediante gráficas interactivas y KPIs claros.
- Detectar patrones de mejora a través del análisis de datos históricos.
- Persistir los datos localmente para consulta offline y seguimiento temporal.

La aplicación consume datos en tiempo real desde la API pública de HenrikDev, los procesa y los almacena en una base de datos local, ofreciendo una interfaz moderna tipo dashboard gaming.

2. Arquitectura del Sistema

2.1. Descripción de la arquitectura

El sistema está compuesto por tres capas principales:

- Frontend desarrollado con React y Vite.
- Backend desarrollado con FastAPI y Python.
- Base de datos PostgreSQL para la persistencia de la información.

El usuario interactúa con la interfaz web desde su navegador. El frontend se comunica mediante peticiones HTTP con el backend, el cual consulta la API de HenrikDev para obtener información actualizada de los jugadores de Valorant. Posteriormente, los datos son almacenados en PostgreSQL para futuras consultas y análisis.

2.2. Flujo de datos

1. El usuario ingresa un Riot Name y Riot Tag en el formulario de búsqueda.
2. El frontend envía una petición GET al backend.
3. El backend consulta la HenrikDev API para obtener los datos de la cuenta y el rango competitivo.
4. Si la consulta es exitosa, los datos se almacenan en PostgreSQL.

5. La respuesta se devuelve al frontend para mostrarla al usuario.
6. El usuario puede consultar el dashboard o el historial de búsquedas almacenadas.

2.3. Modelo de datos

La base de datos está compuesta principalmente por dos entidades:

Players

- ID
- PUUID
- Riot Name
- Riot Tag
- Región
- Nivel de cuenta
- Fecha de actualización
- Fecha de creación

Player Stats

- ID
- ID del jugador
- Rango actual
- Tier del rango
- RR
- Rango máximo alcanzado
- KDA
- Win Rate
- Headshot Percentage
- Fecha de actualización

La relación entre ambas entidades es de uno a muchos.

3. Tecnologías Utilizadas

3.1. Backend

- Python 3.12
- FastAPI
- SQLAlchemy
- Asyncpg
- Pydantic
- Pydantic Settings
- Alembic
- HTTPX
- Uvicorn

3.2. Frontend

- React
- Vite
- TailwindCSS
- Axios
- React Router
- Recharts
- Google Fonts (Inter y Poppins)

3.3. Base de Datos

- PostgreSQL 16
- Asyncpg

3.4. Infraestructura

- Docker
- Docker Compose
- WSL 2
- Debian

3.5. API Externa

- HenrikDev Valorant API
- Riot Games API

4. Desarrollo e Implementación

4.1. Estructura del proyecto

El proyecto se divide en tres módulos principales:

Backend

Contiene los endpoints de la API, la configuración del sistema, modelos ORM, esquemas de validación, servicios para consumir la API de HenrikDev y servicios de persistencia en PostgreSQL.

Frontend

Incluye las páginas principales, componentes reutilizables, sistema de rutas, servicios de consumo de API y estilos de la interfaz.

Base de Datos

Contiene los scripts de inicialización y la estructura de PostgreSQL.

Además, el proyecto cuenta con Docker Compose para la orquestación de todos los servicios.

4.2. Endpoints de la API

- **GET /api/health** → Verifica el estado del sistema.
- **GET /api/player/{name}/{tag}** → Consulta un jugador y almacena la información.
- **GET /api/player/{name}/{tag}/matches** → Obtiene el historial de partidas.
- **GET /api/players** → Lista todos los jugadores almacenados.
- **GET /api/players/{id}** → Obtiene la información de un jugador específico.

4.3. Variables de entorno

Backend:

- Usuario PostgreSQL
- Contraseña PostgreSQL
- Host PostgreSQL
- Puerto PostgreSQL
- Nombre de la base de datos
- Configuración de CORS
- API Key de HenrikDev

Frontend:

- URL base de la API

4.4. Despliegue

El sistema puede ejecutarse mediante Docker Compose, levantando automáticamente los contenedores de:

- Frontend
- Backend
- PostgreSQL

También puede ejecutarse en modo desarrollo utilizando FastAPI y React por separado.

Las migraciones de base de datos son gestionadas mediante Alembic.

5. Conclusiones

5.1. Resultados obtenidos

- Se desarrolló una plataforma funcional para consultar jugadores de Valorant y visualizar estadísticas en tiempo real.
- Se implementó una arquitectura limpia y escalable con separación de responsabilidades.
- Se logró la persistencia de datos utilizando PostgreSQL.
- Se diseñó una interfaz moderna orientada a jugadores competitivos.
- Se realizó la contenerización completa utilizando Docker y Docker Compose.
- Se implementaron KPIs y visualizaciones gráficas para el análisis del rendimiento.

5.2. Posibles mejoras futuras

1. Implementar autenticación de usuarios.
2. Agregar estadísticas avanzadas y análisis predictivos.
3. Incorporar historial completo de MMR.
4. Añadir análisis detallado por agente.
5. Comparar estadísticas entre jugadores.
6. Generar recomendaciones automáticas de mejora.
7. Permitir exportación de datos.
8. Implementar notificaciones.
9. Agregar soporte multilenguaje.
10. Incorporar pruebas automatizadas.

ValoMetrics — Proyecto Universitario

Arquitectura de Software | FastAPI + React + PostgreSQL + Docker

Mayo 2026